Mocha 完整教程

基于 mochajs.org 官方文档 · 中文版 · 2026

目录

1. Mocha 简介
2. 安装
3. 快速开始
4. 测试结构
5. 钩子（Hooks）
6. 待定测试（Pending Tests）
7. 独占测试（only）
8. 跳过测试（skip）
9. 重试测试
10. 异步测试
11. 同步测试
12. 箭头函数
13. 报告器（Reporters）
14. 配置文件
15. 命令行选项
16. 接口（Interfaces）
17. 断言库
18. 超时设置
19. 差异输出
20. 测试覆盖率
21. 最佳实践

1. Mocha 简介

Mocha 是一个功能丰富的 JavaScript 测试框架，运行在 Node.js 和浏览器中，让异步测试变得简单而有趣。Mocha 按顺序串行运行测试，提供灵活准确的报告，并将未捕获的异常映射到正确的测试用例。

主要特性：

- 支持浏览器和 Node.js
- 支持异步测试（包括 Promise 和 async/await）
- 可配置的测试超时
- 支持多种断言库（assert、chai、should.js 等）
- 丰富的报告器（spec、dot、nyan、tap 等）
- 支持 before/after/beforeEach/afterEach 钩子
- 支持 only/skip 控制测试执行
- 支持测试重试

2. 安装

全局安装

```
npm install --global mocha
```

作为开发依赖安装（推荐）

```
npm install --save-dev mocha
```

验证安装

```
npx mocha --version
```

> 推荐将 mocha 安装为项目的 devDependency，并通过 `npx mocha` 或 npm scripts 运行，避免全局版本冲突。

3. 快速开始

创建项目结构：

```
mkdir my-project && cd my-project
npm init -y
npm install --save-dev mocha
```

创建测试文件 `test/test.mjs`：

```
import assert from 'node:assert/strict';

describe('Array', function () {
  describe('#indexOf()', function () {
    it('should return -1 when the value is not present', function () {
      assert.equal([1, 2, 3].indexOf(4), -1);
    });
  });
});
```

在 `package.json` 中添加测试脚本：

```
{
  "scripts": {
    "test": "mocha"
  }
}
```

运行测试：

```
npm test
```

输出示例：

```
  Array
    #indexOf()
      ✓ should return -1 when the value is not present


  1 passing (9ms)
```

4. 测试结构

Mocha 使用 `describe` 和 `it` 来组织测试：

- `describe(name, fn)`：定义测试套件（可嵌套）
- `it(name, fn)`：定义单个测试用例
- `context`：`describe` 的别名
- `specify`：`it` 的别名

```
describe('Calculator', function () {
  describe('add()', function () {
    it('should return 3 when adding 1 and 2', function () {
      assert.equal(add(1, 2), 3);
    });

    it('should return 0 when adding -1 and 1', function () {
      assert.equal(add(-1, 1), 0);
    });
  });

  describe('subtract()', function () {
    it('should return 1 when subtracting 1 from 2', function () {
      assert.equal(subtract(2, 1), 1);
    });
  });
});
```

5. 钩子（Hooks）

Mocha 提供四种钩子，用于在测试前后执行设置和清理操作：

钩子	执行时机
`before()`	当前 describe 块中所有测试运行之前执行一次
`after()`	当前 describe 块中所有测试运行之后执行一次
`beforeEach()`	当前 describe 块中每个测试运行之前执行
`afterEach()`	当前 describe 块中每个测试运行之后执行

```
describe('Database', function () {
  let db;

  before(function () {
    // 在所有测试前连接数据库
    db = connectToDatabase();
  });

  after(function () {
    // 在所有测试后断开连接
    db.disconnect();
  });

  beforeEach(function () {
    // 每个测试前清空数据
    db.clear();
  });

  afterEach(function () {
    // 每个测试后记录日志
    console.log('test finished');
  });

  it('should save a record', function () {
    db.save({ id: 1 });
    assert.equal(db.count(), 1);
  });
});
```

根级钩子（Root Hooks）

在所有 describe 块外定义的钩子称为根级钩子，作用于所有测试文件：

```
// test/hooks.mjs
export const mochaHooks = {
  beforeAll() {
    // 在所有测试套件之前运行一次
  },
  afterAll() {
    // 在所有测试套件之后运行一次
  },
  beforeEach() {
    // 在每个测试之前运行
  },
  afterEach() {
    // 在每个测试之后运行
  }
};
```

在配置文件中注册：

```
// .mocharc.cjs
module.exports = {
  require: ['test/hooks.mjs']
};
```

6. 待定测试（Pending Tests）

没有回调函数的测试被视为"待定"（pending），用于标记尚未实现的测试：

```
describe('Array', function () {
  describe('#indexOf()', function () {
    // 待定测试 - 没有回调
    it('should return -1 when the value is not present');
  });
});
```

待定测试会在报告中显示为 pending 状态，不会导致测试失败。

7. 独占测试（only）

使用 `.only` 可以只运行指定的测试套件或测试用例：

```
// 只运行这个 describe 块
describe.only('Array', function () {
  it('should work', function () { /* ... */ });
});

// 只运行这个 it
describe('Array', function () {
  it.only('should return -1', function () {
    assert.equal([1,2,3].indexOf(4), -1);
  });

  it('this test will not run', function () { /* ... */ });
});
```

> 注意：`.only` 仅用于开发调试，提交代码前应移除，否则 CI 中其他测试不会运行。

可以链式使用多个 `.only`：

```
describe('Array', function () {
  it.only('test 1', function () { /* runs */ });
  it.only('test 2', function () { /* runs */ });
  it('test 3', function () { /* skipped */ });
});
```

8. 跳过测试（skip）

使用 `.skip` 跳过指定的测试套件或测试用例：

```
// 跳过整个 describe 块
describe.skip('broken feature', function () {
  it('should work', function () { /* skipped */ });
});
```

```
// 跳过单个测试
describe('Array', function () {
  it.skip('should return -1', function () {
    // 这个测试会被跳过
  });
});
```

也可以在测试内部动态跳过：

```
it('should only run on macOS', function () {
  if (process.platform !== 'darwin') {
    this.skip(); // 动态跳过
  }
  // macOS 特定测试...
});
```

9. 重试测试

对于不稳定的测试（如网络请求），可以设置重试次数：

```
describe('flaky network tests', function () {
  this.retries(4); // 最多重试4次

  it('should eventually succeed', function () {
    // 不稳定的测试
  });
});
```

也可以对单个测试设置重试：

```
it('should connect to server', function () {
  this.retries(3);
  // ...
});
```

> 重试功能适用于集成测试和端到端测试，单元测试应避免使用，因为不稳定的单元测试通常意

> 味着代码有问题。

10. 异步测试

使用回调（done）

```
describe('User', function () {
  it('should save async', function (done) {
    User.save({ name: 'Alice' }, function (err) {
      if (err) return done(err);
      done();
    });
  });
});
```

使用 Promise

```
it('should fetch user', function () {
  return fetchUser(1).then(function (user) {
    assert.equal(user.name, 'Alice');
  });
});
```

使用 async/await（推荐）

```
it('should fetch user', async function () {
  const user = await fetchUser(1);
  assert.equal(user.name, 'Alice');
});
```

> async/await 是最简洁的异步测试写法，推荐优先使用。

11. 同步测试

同步测试省略回调参数即可，Mocha 会自动继续执行下一个测试：

```
describe('Array', function () {
  it('should return -1 when not found', function () {
    assert.equal([1, 2, 3].indexOf(4), -1);
  });
});
```

12. 箭头函数

Mocha 不推荐使用箭头函数，因为箭头函数会绑定外部的 `this`，导致无法访问 Mocha 上下文（如 `this.timeout()`、`this.skip()`、`this.retries()`）：

```
// 不推荐
it('should work', () => {
  this.timeout(1000); // 错误! this 不是 Mocha 上下文
});

// 推荐
it('should work', function () {
  this.timeout(1000); // 正确
});
```

> 如果不需要使用 Mocha 上下文，箭头函数可以使用，但为了一致性，建议统一使用普通函数。

13. 报告器（Reporters）

通过 `--reporter` 选项指定报告器：

```
npx mocha --reporter spec    # 默认，详细输出
npx mocha --reporter dot     # 简洁点状输出
npx mocha --reporter min     # 最小化输出
npx mocha --reporter tap     # TAP 格式
npx mocha --reporter json    # JSON 格式
npx mocha --reporter nyan    # 彩虹猫（有趣）
```

报告器	说明
spec	默认，层级缩进显示所有测试
dot	每个测试显示一个点，失败显示 F
min	只显示摘要和失败信息
tap	TAP 协议格式，适合 CI 集成
json	JSON 格式，适合程序解析
json-stream	流式 JSON，每行一个事件
landing	飞机降落动画
nyan	彩虹猫动画

第三方报告器

```
npm install --save-dev mochawesome
npx mocha --reporter mochawesome
```

14. 配置文件

Mocha 支持多种配置文件格式，按优先级查找：

- `.mocharc.cjs` / `.mocharc.js`
- `.mocharc.yaml` / `.mocharc.yml`
- `.mocharc.json` / `.mocharc.jsonc`
- `package.json` 中的 `mocha` 字段

.mocharc.json 示例

```
{
  "spec": "test/**/*.spec.js",
```

```
  "reporter": "spec",
  "timeout": 5000,
  "require": ["./test/setup.js"],
  "recursive": true
}
```

.mocharc.yml 示例

```
spec: test/**/*.spec.js
reporter: spec
timeout: 5000
require:
  - ./test/setup.js
recursive: true
```

package.json 中配置

```
{
  "mocha": {
    "spec": "test/**/*.spec.js",
    "timeout": 5000,
    "reporter": "spec"
  }
}
```

15. 命令行选项

选项	说明
`--spec <pattern>`	指定测试文件（默认 test/*.{js,mjs,cjs}）
`--reporter <name>`	指定报告器
`--timeout <ms>`	设置超时时间（默认 2000ms）
`--recursive`	递归查找子目录中的测试文件

`--require <module>`	在测试前 require 指定模块
`--watch` / `-w`	监听文件变化，自动重新运行
`--grep <pattern>`	只运行匹配正则的测试
`--fgrep <string>`	只运行包含指定字符串的测试
`--invert`	反转 --grep/--fgrep 的匹配
`--bail`	第一个失败后停止运行
`--exit`	测试完成后强制退出进程
`--parallel`	并行运行测试文件
`--retries <n>`	失败测试重试次数
`--no-timeouts`	禁用超时

常用示例

```
# 只运行包含 "login" 的测试
npx mocha --grep "login"

# 监听模式
npx mocha --watch

# 递归运行所有测试
npx mocha --recursive

# 并行运行（加速大型测试套件）
npx mocha --parallel
```

16. 接口（Interfaces）

Mocha 支持多种测试接口风格：

BDD（默认）

```
describe('Array', function () {
  before(function () { /* ... */ });
  it('should work', function () { /* ... */ });
});
```

TDD

```
suite('Array', function () {
  suiteSetup(function () { /* before all */ });
  suiteTeardown(function () { /* after all */ });
  setup(function () { /* before each */ });
  teardown(function () { /* after each */ });
  test('indexOf', function () { /* ... */ });
});
```

Exports

```
module.exports = {
  before: function () { /* ... */ },
  'Array': {
    '#indexOf()': {
      'should return -1': function () {
        assert.equal([1,2,3].indexOf(4), -1);
      }
    }
  }
};
```

QUnit

```
suite('Array');
test('#indexOf()', function () {
  assert.equal([1,2,3].indexOf(4), -1);
});
```

通过 `--ui` 选项切换接口：

```
npx mocha --ui tdd
```

17. 断言库

Mocha 不内置断言库，可以搭配任意断言库使用：

Node.js 内置 assert

```
import assert from 'node:assert/strict';

it('should equal', function () {
  assert.equal(1 + 1, 2);
  assert.deepEqual({ a: 1 }, { a: 1 });
  assert.throws(() => { throw new Error('oops'); });
});
```

Chai（推荐）

```
import { expect, should, assert } from 'chai';

// expect 风格
expect(1 + 1).to.equal(2);
expect([1, 2, 3]).to.include(2);
expect('hello').to.be.a('string');

// should 风格
should();
(1 + 1).should.equal(2);

// assert 风格
assert.equal(1 + 1, 2);
assert.isArray([1, 2, 3]);
```

Should.js

```
import 'should';
(1 + 1).should.equal(2);
```

Expect.js

```
import expect from 'expect.js';
expect(1 + 1).to.equal(2);
```

18. 超时设置

默认超时为 2000ms，可以在不同级别设置：

全局设置

```
npx mocha --timeout 5000
```

套件级别

```
describe('slow tests', function () {
  this.timeout(5000); // 5秒

  it('should finish in time', function (done) {
    setTimeout(done, 3000);
  });
});
```

单个测试

```
it('should be slow', function (done) {
  this.timeout(10000); // 10秒
  setTimeout(done, 8000);
});
```

禁用超时

```
this.timeout(0); // 0 表示禁用超时
```

19. 差异输出

当断言失败时，Mocha 会自动显示期望值和实际值的差异：

```
AssertionError: expected { a: 1, b: 2 } to deeply equal { a: 1, b: 3 }
  + expected - actual

   {
     "a": 1,
  -  "b": 2
  +  "b": 3
   }
```

通过 `--no-diff` 可以禁用差异输出。

20. 测试覆盖率

使用 `nyc` （Istanbul 的命令行工具）生成覆盖率报告：

```
npm install --save-dev nyc
```

在 `package.json` 中配置：

```
{
  "scripts": {
    "test": "mocha",
    "coverage": "nyc mocha"
  },
  "nyc": {
    "reporter": ["text", "html", "lcov"],
    "include": ["src/**/*.js"],
```

```
    "exclude": ["test/**/*.js"]
  }
}
```

运行覆盖率：

```
npm run coverage
```

输出示例：

```
----------|---------|----------|---------|---------|
File      | % Stmts | % Branch | % Funcs | % Lines |
----------|---------|----------|---------|---------|
All files |   85.71 |    66.67 |     100 |   85.71 |
 index.js |   85.71 |    66.67 |     100 |   85.71 |
----------|---------|----------|---------|---------|
```

21. 最佳实践

- **测试文件命名**：使用 `*.spec.js` 或 `*.test.js` 后缀，放在 `test/` 目录下
- **每个测试只验证一件事**：保持测试简单、聚焦
- **使用描述性名称**：`it('should return -1 when value not found')` 而非 `it('test1')`
- **避免测试间依赖**：每个测试应独立运行，使用 beforeEach 重置状态
- **不要在生产代码中留 .only**：提交前检查并移除
- **合理设置超时**：异步测试设置足够的超时时间
- **使用 async/await**：比回调和 Promise 链更清晰
- **分离测试关注点**：单元测试、集成测试、端到端测试分开组织

推荐项目结构

```
my-project/
├── src/
│   ├── calculator.js
│   └── user.js
├── test/
│   ├── unit/
```

```
│   │   ├── calculator.spec.js
│   │   └── user.spec.js
│   ├── integration/
│   │   └── api.spec.js
│   └── hooks.js
├── .mocharc.json
└── package.json
```

完整示例

```
// src/calculator.js
export function add(a, b) { return a + b; }
export function divide(a, b) {
  if (b === 0) throw new Error('Division by zero');
  return a / b;
}

// test/calculator.spec.js
import { expect } from 'chai';
import { add, divide } from '../src/calculator.js';

describe('Calculator', function () {
  describe('add()', function () {
    it('should add two positive numbers', function () {
      expect(add(1, 2)).to.equal(3);
    });

    it('should handle negative numbers', function () {
      expect(add(-1, 1)).to.equal(0);
    });
  });

  describe('divide()', function () {
    it('should divide correctly', function () {
      expect(divide(10, 2)).to.equal(5);
    });

    it('should throw on division by zero', function () {
      expect(() => divide(1, 0)).to.throw('Division by zero');
    });
```

```
    });
});
```

Mocha 完整教程 · 基于 mochajs.org 官方文档 · 2026